

מודול 4 — סקריפטינג — תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1. 🎯 תרגילים — מודול 4: סקריפטינג (Python ו-Bash)

2. ✅ פתרונות מלאים — מודול 4: סקריפטינג

🎯 תרגילים — מודול 4: סקריפטינג (Python ו-Bash)

כל הקוד רץ במעבדה שלך. סריקות — רק על 127.0.0.1 או מכונות המעבדה. פתרונות ב- `solutions.md`.

🚦 לפני שמתחילים — קרא אותי!

- מה צריך: מכונת Kali עם עורך (nano או VS Code). כל הקוד שתצטרך מופיע ב- `README.md` — אתה מעתיק, מריץ, ומבין.
- איך עובדים: בתרגילים הקלים (🟢) יש את הקוד המדויק להקליד. בקשים יותר יש 💡 רמז שמפנה לקוד ב-`README`, ו-✅ קריטריון.
- נתקעת? שגיאת Python/Bash תמיד מציינת מספר שורה — קרא אותה. נסה, ואז הצץ ב- `solutions.md` שיש בו את הקוד המלא.
- טיפ: אל תפחד מ"תכנות". כאן זה פשוט — משתנים, תנאים, לולאות. תעתיק, תריץ, תשנה, תבין.

מקרא רמות: 🟢 קל · 🟡 בינוני · 🟠 מתקדם · 🔴 אתגר

חלק א' — Bash (🟢🟡)

4.1 🟢 — כתוב `hello.sh` שמדפיס "Hello Hacker", הפוך אותו לניתן-הרצה, והרץ.

```
# והדבק nano hello.sh צור קובץ עם:
#!/bin/bash
echo "Hello Hacker"
# ואז (Ctrl+0, Enter, Ctrl+X), שמור:
chmod +x hello.sh
./hello.sh
```

👁️ חפש: הפלט Hello Hacker. ✅ הצלחה: הסקריפט רץ והדפיס את המשפט.

4.2 🟢 — סקריפט שמקבל שם עם `read` ומדפיס `Welcome, <name>`.

```
#!/bin/bash
read -p "Enter name: " name
echo "Welcome, $name"
```

✓ הצלחה: הסקריפט שאל שם והדפיס ברכה עם השם.

4.3 — סקריפט שמקבל פורט כארגומנט (\$1) ומדפיס אם הוא 80/443/Unknown. רמז: README — סעיף תנאים ב-Bash (if / elif / else). הרץ עם 80 script.sh ./ . הצלחה: הסקריפט מזהה 80=HTTPS, 443=HTTP, אחר=Unknown.

4.4 — לולאה ב-Bash שמדפיסה Pinging 10.0.0.X עבור X מ-1 עד 5. רמז: README — סעיף לולאות (for i in \$(seq 1 5)). הצלחה: 5 שורות פלט עם X מ-1 עד 5. ✓

חלק ב' — Python בסיסי (●●)

4.5 — סקריפט Python שמגדיר target ומדפיס אותו עם f-string.

```
target = "10.0.0.5"
print(f"Scanning target: {target}")
```

✓ הצלחה: הפלט Scanning target: 10.0.0.5.

4.6 — נתון banner = "Apache/2.4.49 (Debian)". חלץ והדפס רק את 2.4.49. רמז: README — סעיף מחרוזות. השתמש ב- split("/") ואז split(" ") . הצלחה: הפלט הוא 2.4.49 בלבד. ✓

4.7 — צור מילון services הממפה 21,22,80,443 לשמות, ובלולאה הדפס Port X -> Service . רמז: README — סעיף מילונים ולולאות (for port, name in services.items()). הצלחה: 4 שורות Port X -> Service .

חלק ג' — לוגיקה ולולאות (●)

4.8 — פונקציה check_port(port) שמחזירה שם שירות אם מוכר, אחרת "Unknown". רמז: README — סעיף פונקציות. השתמש ב- dict.get(port, "Unknown") . הצלחה: check_port(80) מחזיר "HTTP", check_port(9999) מחזיר "Unknown".

4.9 — לולאה על פורטים 20–25 שמדפיסה לכל אחד את שם השירות (בעזרת 4.8). 💡 רמז: `for port in range(20, 26)` — שים לב ש-`range` לא כולל את הקצה העליון. ✅ הצלחה: 6 שורות, אחת לכל פורט 20–25.

חלק ד' — הכלים האמיתיים (מתקדם) 🟡

4.10 — הרץ את ה-**Ping Sweep** (מהחומר) על תת-הרשת שלך. 💡 רמז: README — קוד ה-Ping Sweep המלא. החלף את הרשת לשלך. ✅ הצלחה: הסקריפט הדפיס אילו מכונות "חיות".

4.11 — הרץ את ה-**Port Scanner** (מהחומר) על 127.0.0.1.

```
# בטרמינל נפרד, פתח פורט לבדיקה
python3 -m http.server 8000
# ואז הרץ את הסורק שלך על 127.0.0.1
```

👁️ **חפש:** הסורק אמור לדווח שפורט 8000 (ואולי אחרים) פתוח. ✅ הצלחה: הסורק זיהה לפחות פורט פתוח אחד.

4.12 — קרא את קוד ה-Port Scanner וענה: מה מחזירה `connect_ex` בפורט פתוח? מה תפקיד `settimeout`? למה צריך `KeyboardInterrupt` ? try/except? 💡 רמז: README — הסבר קוד הסורק. ✅ הצלחה: `connect_ex` = 0 בפורט פתוח; `settimeout` מונע תקיעה; ה-`except` מאפשר עצירה נקייה ב-Ctrl+C.

🔴 אתגר מסכם — "שדרוג הסורק"

שדרג את ה-Port Scanner כך שיכלול:

1. **שם שירות** — לצד כל פורט פתוח, הדפס את שם השירות ממילון `services`.
2. **טווח מהמשתמש** — קבל פורט התחלה וסיום מהמשתמש (`input`).
3. **סיכום** — בסוף: `[*] Scan complete. Found N open ports.`
4. **זמן** — מדוד את משך הסריקה עם המודול `time`.
5. **(בונוס Bash)** — סקריפט Bash שמריץ קודם Ping Sweep, ואז Port Scanner על כל מכונה חיה.

פלט רצוי לדוגמה:

```
[*] Enter target: 127.0.0.1
[*] Ports 1-1024
[+] Port 80 is OPEN (HTTP)
[+] Port 8000 is OPEN
[*] Scan complete. Found 2 open ports in 2.3s
```

💡 רמז: התחל מקוד הסורק ב-README והוסף רכיב אחד בכל פעם. פתרון מלא ב- `solutions.md`.
הצלחה: סורק משודרג שמדפיס שמות שירות, מקבל טווח, ומסכם עם זמן.

🏠 בניית שני הכלים = יסוד המעבר לשלב הסריקה (מודול 6), שם נשתמש ב-nmap המקצועי.

✓ רשימת בקרה — לפני מעבר למודול 5

- ☐ אני כותב ומריץ סקריפט Bash עם Shebang, משתנים, תנאים ולולאות
- ☐ בניתי והרצתי את ה-Ping Sweep
- ☐ אני מבין טיפוסים ומבני נתונים ב-Python
- ☐ אני כותב פונקציות ולולאות ב-Python
- ☐ בניתי והרצתי את ה-Port Scanner והבנתי את הקוד
- ☐ אני יודע מתי להשתמש ב-Bash ומתי ב-Python

פתרונות מלאים: `solutions.md`

✓ פתרונות מלאים — מודול 4: סקריפטינג

נסה לבד ב- `missions.md` קודם.

חלק א' — Bash

4.1

```
#!/bin/bash
echo "Hello Hacker"
```

```
chmod +x hello.sh
./hello.sh
```

4.2

```
#!/bin/bash
read -p "Enter name: " name
echo "Welcome, $name"
```

4.3

```
#!/bin/bash
if [ "$1" -eq 80 ]; then
    echo "HTTP"
elif [ "$1" -eq 443 ]; then
    echo "HTTPS"
else
    echo "Unknown"
fi
```

הרצה: `./script.sh 443` → `. HTTPS`

4.4

```
#!/bin/bash
for i in $(seq 1 5); do
    echo "Pinging 10.0.0.$i"
done
```

חלק ב' — Python בסיסי

4.5

```
target = "10.0.0.5"
print(f"[*] Target is {target}")
```

4.6

```
banner = "Apache/2.4.49 (Debian)"
version = banner.split("/")[1].split(" ")[0]
print(version)    # 2.4.49
```

הסבר: `split("/")` → `['Apache', '2.4.49 (Debian)']` ; לוקחים אינדקס 1, ואז `split(" ")` [0] לוקח את החלק לפני הרווח.

4.7

```
services = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS"}
for port, name in services.items():
    print(f"Port {port} -> {name}")
```

חלק ג' — לוגיקה ולולאות

4.8

```
def check_port(port):
    services = {21: "FTP", 22: "SSH", 80: "HTTP", 443: "HTTPS"}
    return services.get(port, "Unknown")
```

`dict.get(key, default)` מחזיר את הערך אם קיים, אחרת את ברירת המחדל.

4.9

```
for port in range(20, 26):
    print(f"Port {port}: {check_port(port)}")
```

חלק ד' — הכלים האמיתיים

4.10 — הרצה:

```
chmod +x ping_sweep.sh
./ping_sweep.sh 192.168.1
```

מספר המכונות החיות תלוי במעבדה שלך — לפחות Kali ו-Gateway אמורים לענות.

4.11


```
# טרמינל 1:
python3 -m http.server 8000
# טרמינל 2:
python3 scanner.py      # 127.0.0.1 הזן
```

תראה Port 8000 is OPEN [+] (ואולי פורטים נוספים שרצים).

4.12 — connect_ex מחזיר 0 כשהפורט פתוח (החיבור הצליח). - settimeout(0.5) מגביל את ההמתנה לחיבור — מונע היתקעות על פורט מסונן ומאיץ את הסריקה. - try/except KeyboardInterrupt מאפשר לעצור בנקישת Ctrl+C בצורה נקייה במקום קריסה מכווערת.

● אתגר מסכם — פתרון מלא

```
#!/usr/bin/env python3
import socket, sys, time

services = {21:"FTP",22:"SSH",80:"HTTP",443:"HTTPS",3389:"RDP",8000:"HTTP-alt"}

target = input("[*] Enter target: ")
start_port = int(input("[*] Start port: "))
end_port = int(input("[*] End port: "))

print(f"[*] Scanning {target} ports {start_port}-{end_port}")
open_count = 0
start_time = time.time()

try:
    for port in range(start_port, end_port + 1):
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(0.5)
        if s.connect_ex((target, port)) == 0:
            name = services.get(port, "")
            print(f"[+] Port {port} is OPEN ({name})" if name else f"[+] Port {port} is OPEN")
            open_count += 1
        s.close()
except KeyboardInterrupt:
    print("\n[!] Stopped by user"); sys.exit()

elapsed = round(time.time() - start_time, 1)
print(f"[*] Scan complete. Found {open_count} open ports in {elapsed}s")
```

בונוס Bash — שרשור Ping Sweep + Scanner:

```
#!/bin/bash
# מוצא מכונות חיות, ואז מריץ את סורק הפיתון על כל אחת
network=$1
for ip in $(seq 1 254); do
    if ping -c 1 -W 1 "$network.$ip" > /dev/null 2>&1; then
        echo "[+] $network.$ip is UP – scanning..."
        echo "$network.$ip" | python3 scanner.py
    fi
done
```

עברת? מצוין — יש לך עכשיו בסיס סקריפטינג אמיתי. המשך למודול 5.